

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Yuke Brilliant Hestiavin - 5024241016

13 November 2026

1 Langkah-Langkah Percobaan

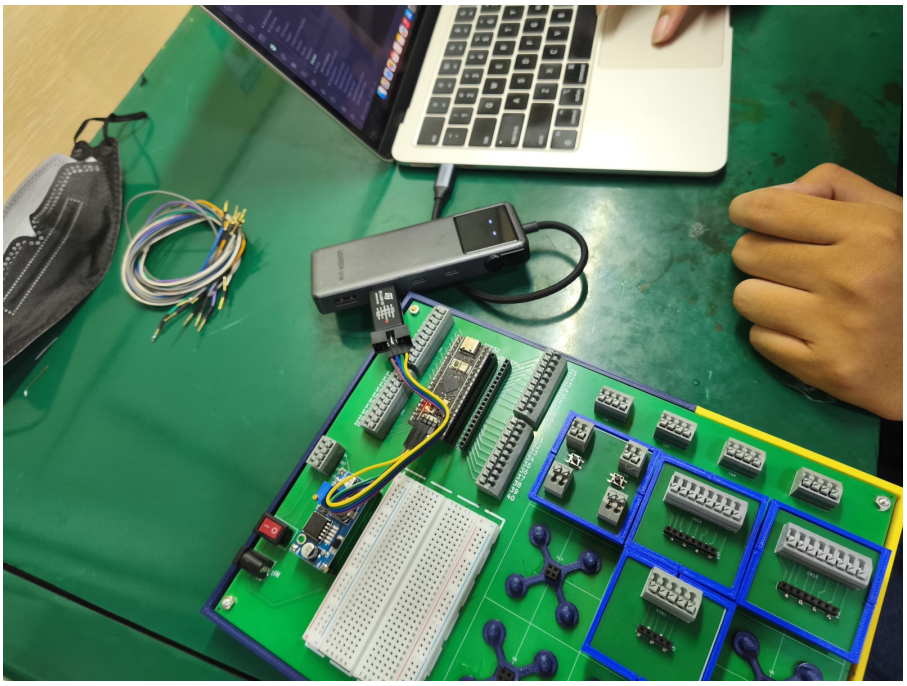
Seluruh percobaan dikerjakan pada papan trainer praktikum yang menyediakan soket untuk board STM32 Blackpill dan ESP32 DevKit, *breadboard*, dan terminal pin. Pemrograman memakai satu proyek PlatformIO di VSCode untuk Percobaan 1 sampai 5. Setiap berganti percobaan, hanya isi `platformio.ini` dan `src/main.cpp` yang diganti, lalu PlatformIO ditunggu sampai selesai memperbarui konfigurasi sebelum *Build* dan *Upload*.

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

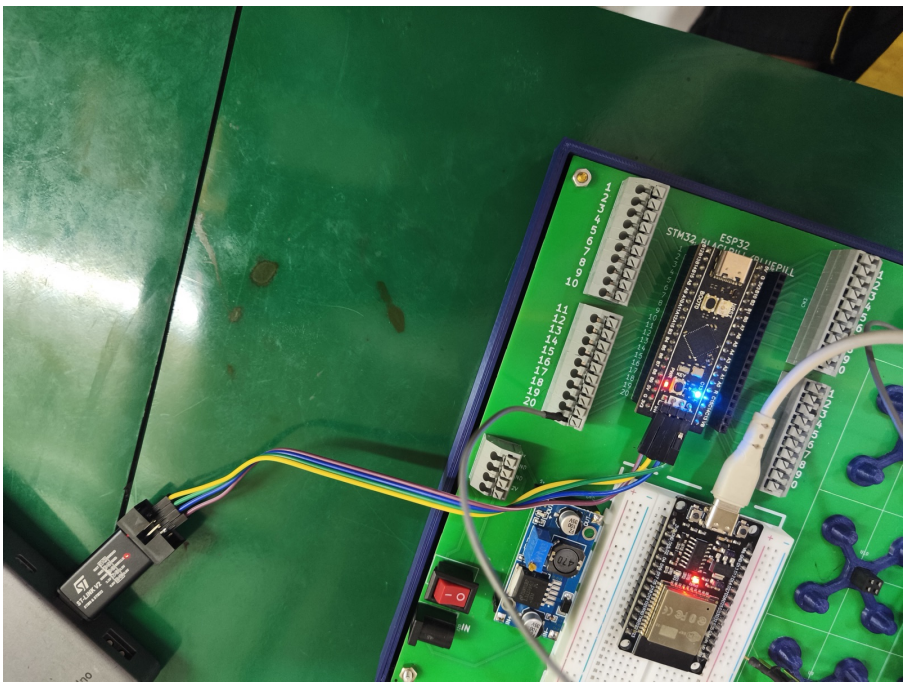
1. Mengisi `platformio.ini` dengan board `blackpill_f401cc`, framework `arduino`, serta `upload_protocol` dan `debug_tool` bernilai `stlink`.
2. Menghubungkan ST-Link V2 ke Blackpill dengan empat kabel, lalu mencolokkan ST-Link ke laptop lewat USB. Pasangan pinnya diperiksa berdasarkan nama pin karena letak pin pada ST-Link bisa berbeda (Tabel 1 dan Gambar 1). ST-Link diatur pada tegangan 3,3 V.
3. Menulis program Blink pada `src/main.cpp` yang mengatur pin PC13 sebagai keluaran, lalu menjalankan *Build* dan *Upload*.
4. Mengamati LED di board (PC13), yang menyala selama 0,5 detik dan mati 0,5 detik secara berulang (Gambar 2).

Pin ST-Link	Pin Blackpill	Fungsi
SWDIO	SWDIO	Jalur data *debug*
SWCLK	SWCLK	Jalur *clock debug*
GND	GND	Ground bersama
3.3V	3V3	Tegangan referensi target

Tabel 1: Sambungan ST-Link ke Blackpill



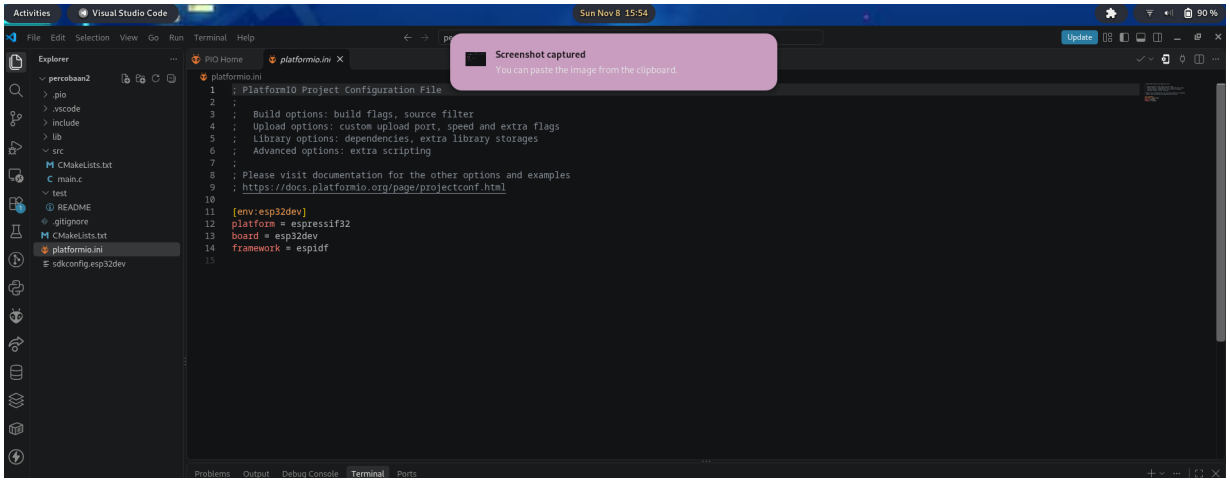
Gambar 1: Rangkaian ST-Link ke Blackpill pada papan trainer



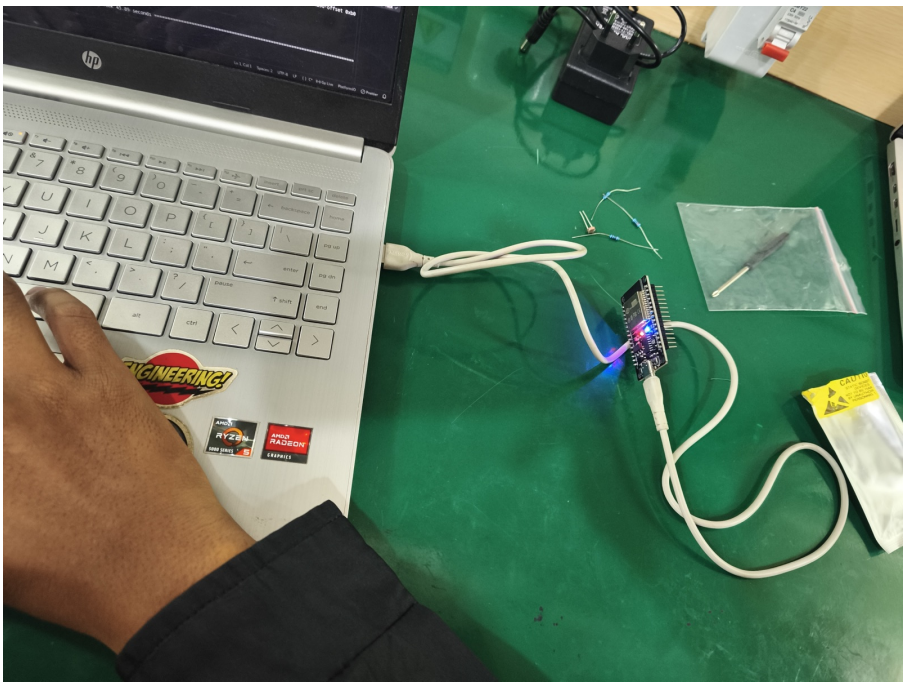
Gambar 2: Blackpill terhubung ke ST-Link, LED pada board menyala setelah program diunggah

1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

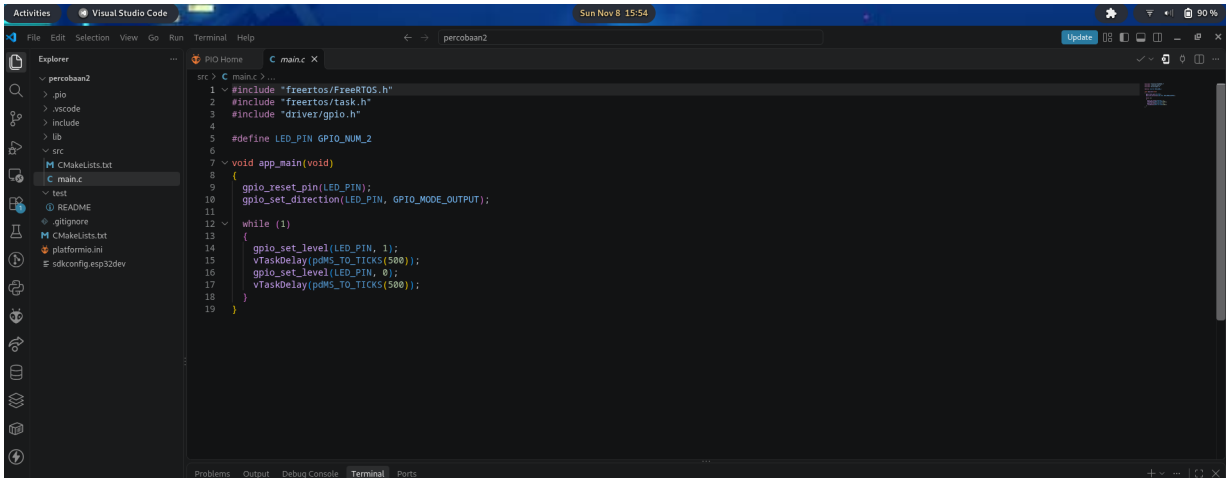
1. Mengganti isi `platformio.ini` menjadi board `esp32dev` dengan framework `espidf` (Gambar 3).
2. Mencolokkan ESP32 ke laptop lewat kabel USB dan memastikan laptop mengenali port serialnya (Gambar 4).
3. Menghapus `src/main.cpp`, membuat `src/main.c`, dan menulis program Blink dengan `app_main()`, `gpio_set_level()`, dan `vTaskDelay()` pada GPIO 2 (Gambar 5). Setelah itu menjalankan *Build* dan *Upload*.
4. Mengamati LED biru pada ESP32 yang berkedip (Gambar 6), lalu membandingkan cara memprogram ESP-IDF dengan Arduino.



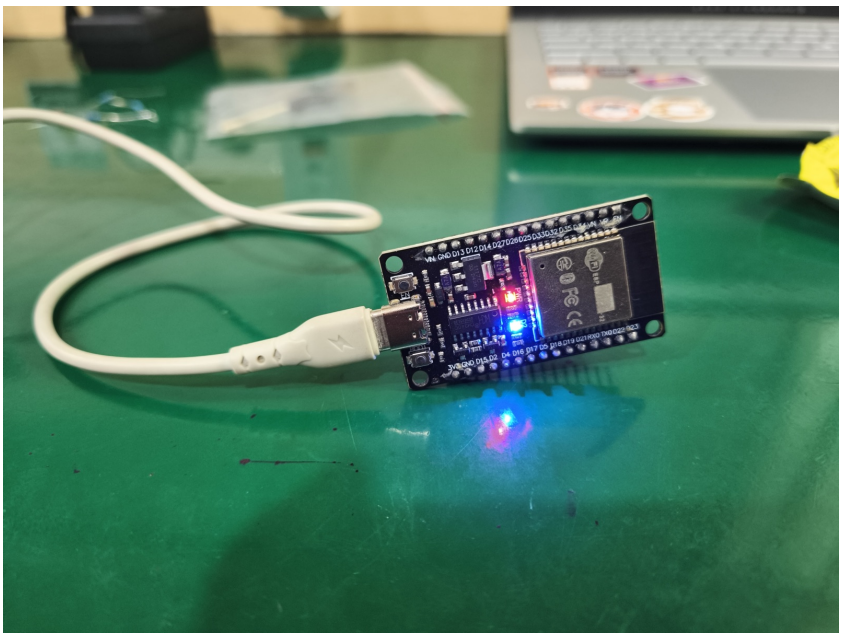
Gambar 3: `platformio.ini` untuk ESP32 dengan framework ESP-IDF



Gambar 4: ESP32 tersambung ke laptop lewat USB



Gambar 5: Kode Blink ESP-IDF pada `src/main.c`



Gambar 6: LED pada ESP32 menyala setelah program diunggah

1.3 Percobaan 3: Pull-up dan Pull-down, Eksternal dan Internal

1. Mengganti `platformio.ini` kembali ke framework Arduino (Gambar 7), menghapus `src/main.c`, dan membuat ulang `src/main.cpp`.

2. **Bagian A (resistor eksternal).** Memasang dua tombol: tombol 1 ke GPIO 32 dengan resistor 10 kΩ ke 3,3 V (*pull-up*) dan kaki lainnya ke GND, tombol 2 ke GPIO 33 dengan resistor 10 kΩ ke GND (*pull-down*) dan kaki lainnya ke 3,3 V (Gambar 8).

3. Menulis Kode Program A (pin dibuat `INPUT` biasa) dan menjalankan *Build* dan *Upload* (Gambar 9). Kedua tombol ditekan bergantian sambil mencatat hasilnya di Serial Monitor (Gambar 10).

4. **Bagian B (resistor internal).** Melepas kedua resistor eksternal dengan tombol tetap terpasang pada GPIO yang sama (Gambar 11). Pasangan kaki tombol dicek agar benar-benar berseberangan.

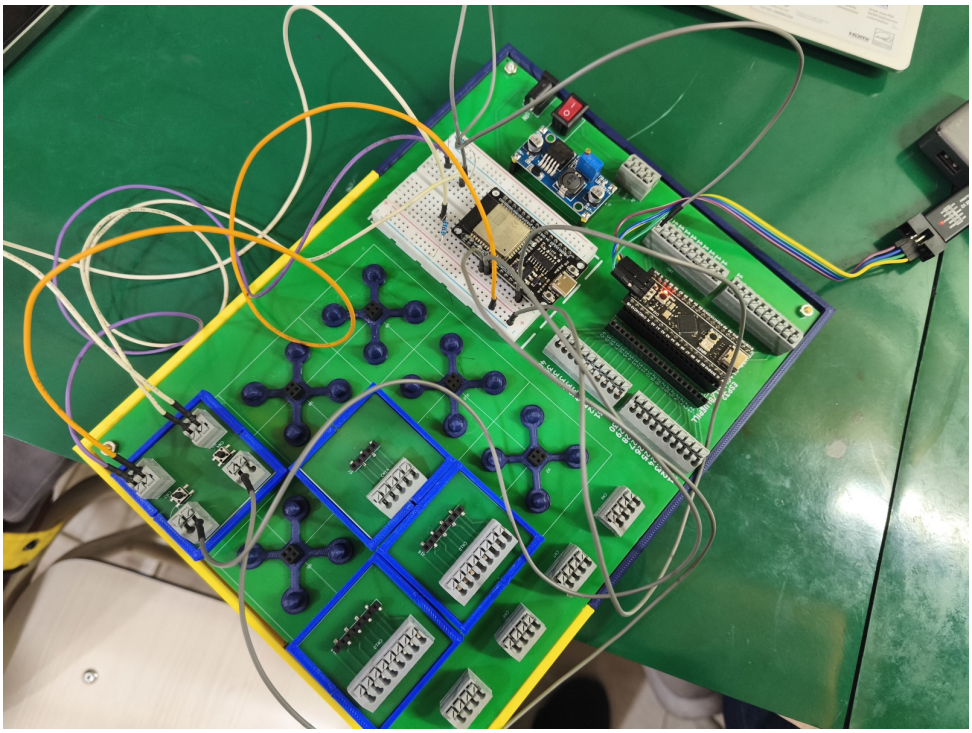
5. Mengganti isi `src/main.cpp` dengan Kode Program B yang memakai `INPUT_PULLUP` pada GPIO 32 dan `INPUT_PULLDOWN` pada GPIO 33 (Gambar 12), lalu *Build*, *Upload*, dan menekan tombol yang sama (Gambar 13).

6. Membandingkan hasil Bagian A dan B. Jika hasilnya terbalik atau berbeda, periksa apakah resistor eksternal sudah benar-benar dilepas.

```
; PlatformIO Project Configuration File
;
;   Build options: build flags, source filter
;   Upload options: custom upload port, speed and extra flags
;   Library options: dependencies, extra library storages
;   Advanced options: extra scripting
;
; Please visit documentation for the other options and examples
; https://docs.platformio.org/page/projectconf.html

[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed=115200
```

Gambar 7: `platformio.ini` untuk ESP32 dengan framework Arduino



Gambar 8: Rangkaian dua tombol dengan resistor *pull-up* dan *pull-down* eksternal



```cpp
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32   // pull-up eksternal
#define BUTTON_PULLDOWN_PIN 33 // pull-down eksternal

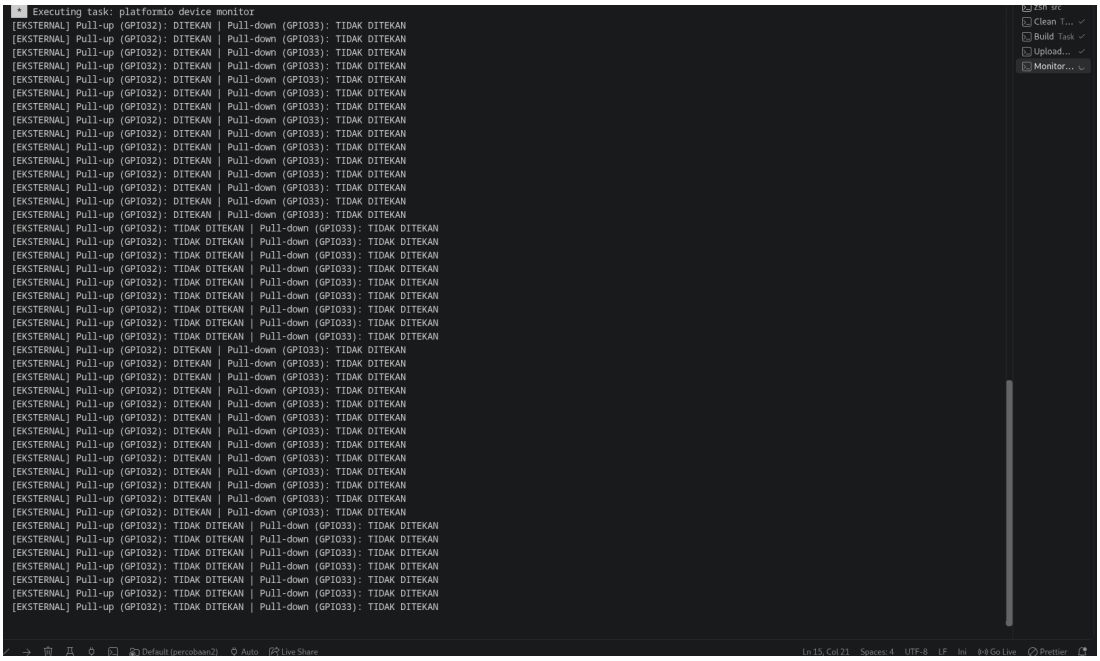
void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT);   // resistor pull-up dipasang eksternal, bukan INPUT_PULLUP internal
  pinMode(BUTTON_PULLDOWN_PIN, INPUT); // resistor pull-down dipasang eksternal
}

void loop() {
  int stateUp = digitalRead
  int stateDown = digitalRe
                                  IN);

  Serial.printf("[EKSTERNAL]                    %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```

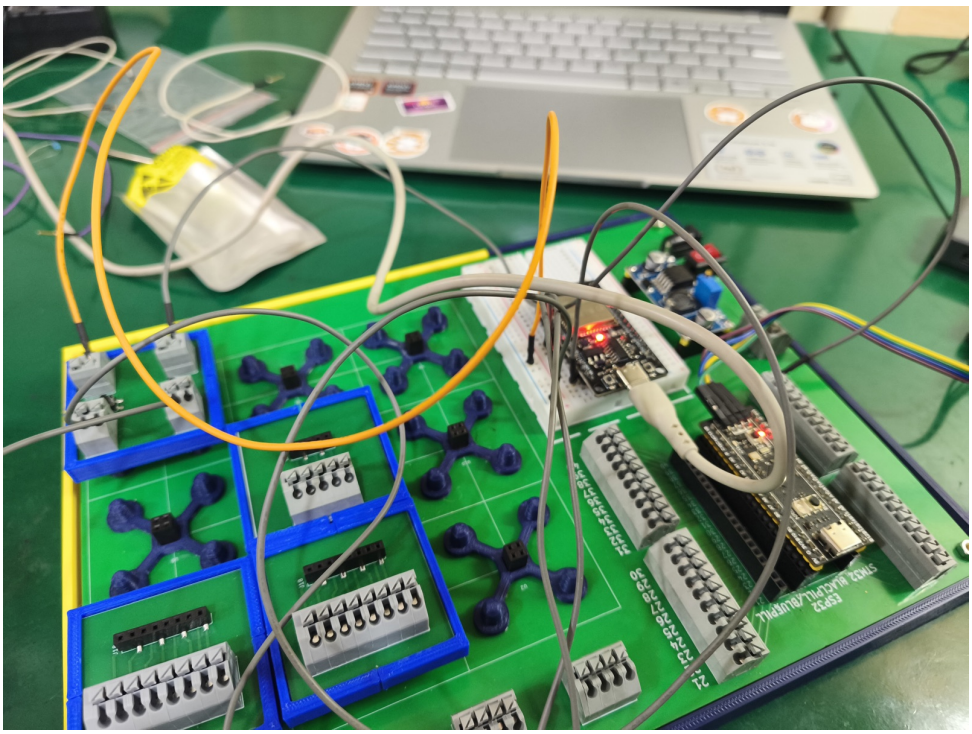
#define LOW 0x0
Expands to:
0x0

Gambar 9: Kode Program A (resistor eksternal)

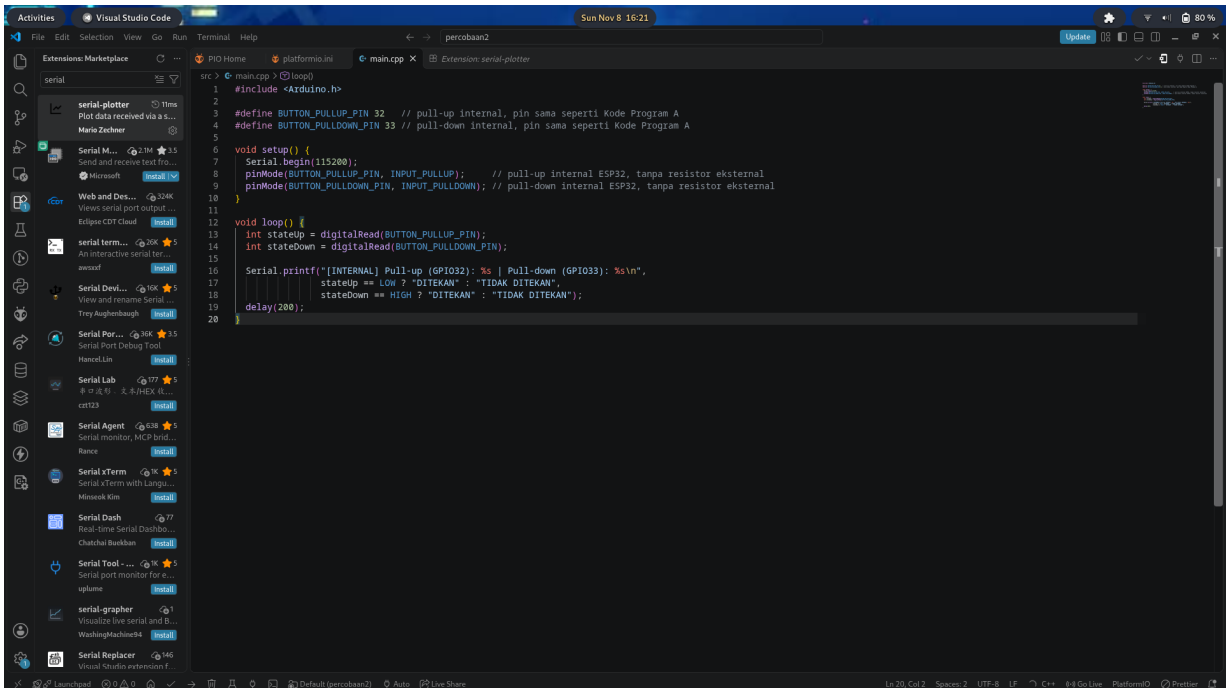


```
Executing task: platformio device monitor
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
```

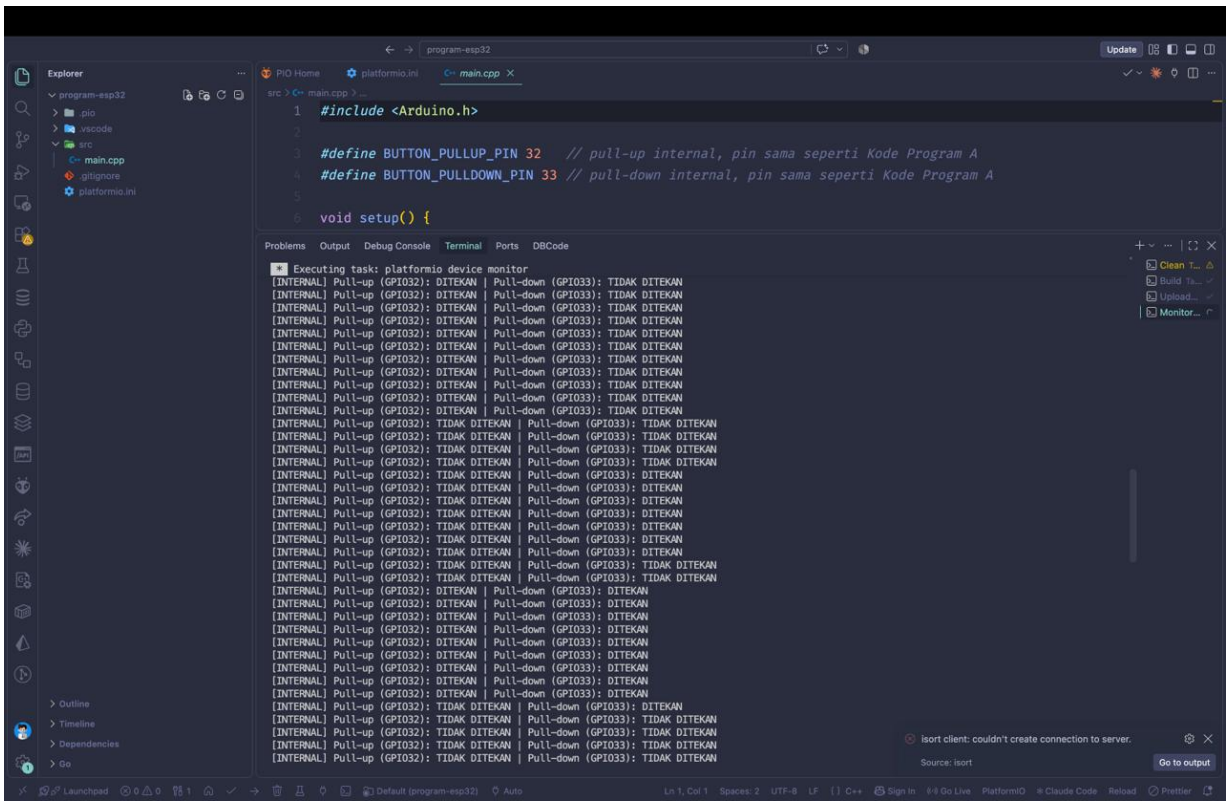
Gambar 10: Serial Monitor Kode Program A



Gambar 11: Rangkaian setelah resistor eksternal dilepas



Gambar 12: Kode Program B (resistor internal)



Gambar 13: Serial Monitor Kode Program B

1.4 Percobaan 4: Debouncing pada Input Tombol

1. Memasang satu tombol pada GPIO 25, kaki lainnya ke GND, dengan mode `INPUT_PULLUP` (Gambar 14).

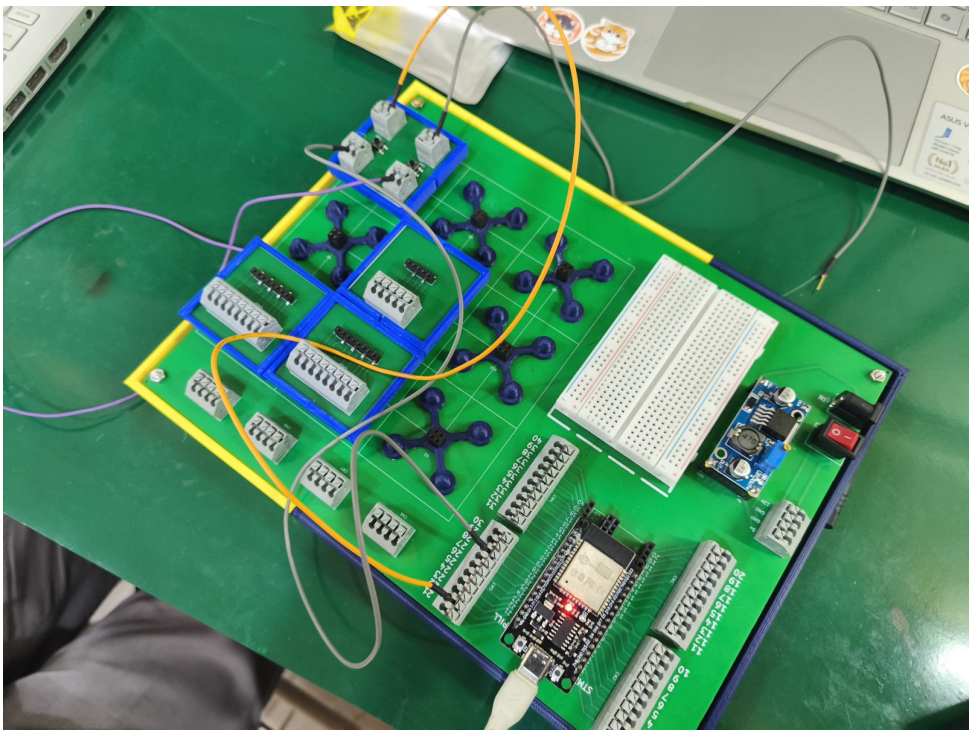
2. Menulis program dengan dua penghitung: `counterNoDebounce` yang menghitung setiap transisi HIGH ke LOW tanpa penyaring, dan `counterWithDebounce` yang hanya menghitung jika keadaan LOW sudah stabil lebih lama dari `DEBOUNCE_DELAY`.

3. Menjalankan *Build* dan *Upload*, lalu menekan tombol satu kali dan membandingkan kedua penghitung di Serial Monitor (Gambar 15).

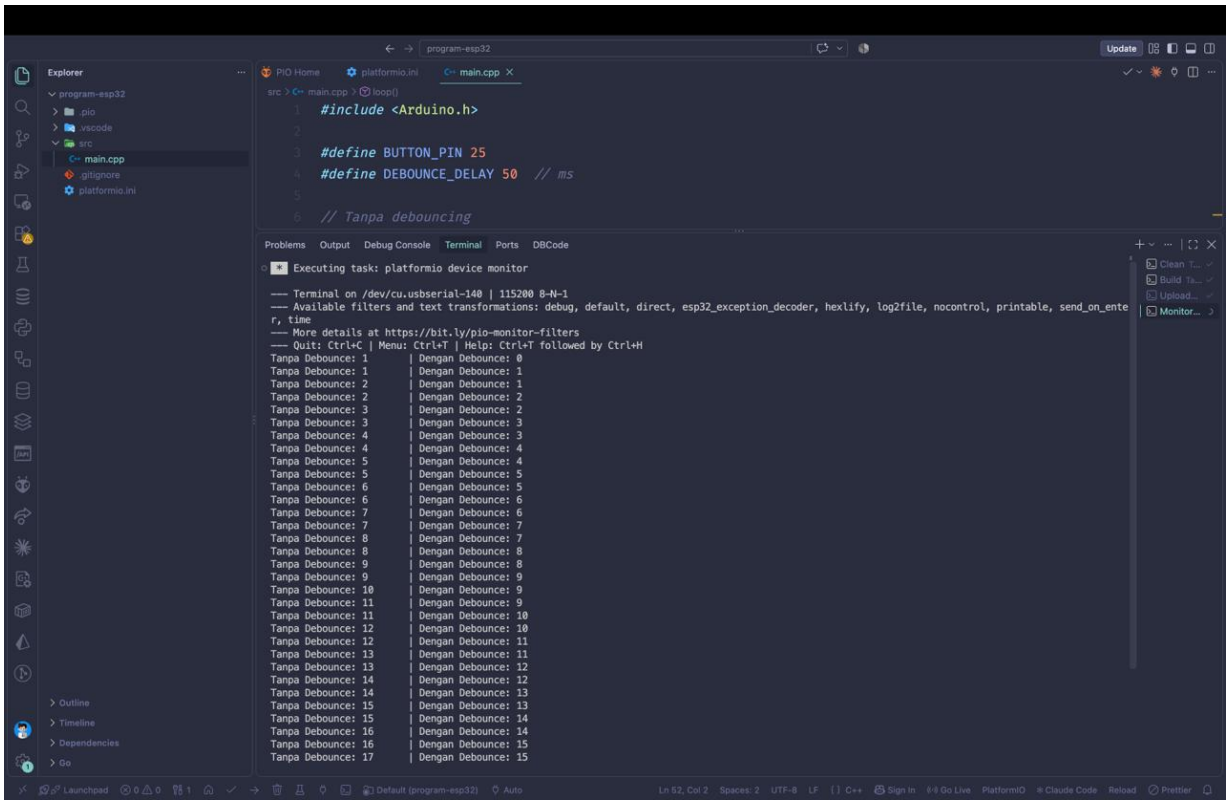
4. Mengulang penekanan dengan cara cepat dan lambat. `counterNoDebounce` bisa naik lebih dari satu untuk satu tekanan, sedangkan `counterWithDebounce` naik tepat satu.

5. Mengubah `DEBOUNCE_DELAY` menjadi 10 ms, *Build* dan *Upload* ulang, lalu menekan tombol lagi. Kemudian mengubahnya menjadi 200 ms dan menekan tombol dengan cepat berulang kali.

6. Membandingkan hasil untuk 10, 50, dan 200 ms.



Gambar 14: Rangkaian satu tombol pada GPIO 25



Gambar 15: Serial Monitor perbandingan penghitung tanpa dan dengan *debouncing* (50 ms)

1.5 Percobaan 5: Pembacaan ADC dengan LDR

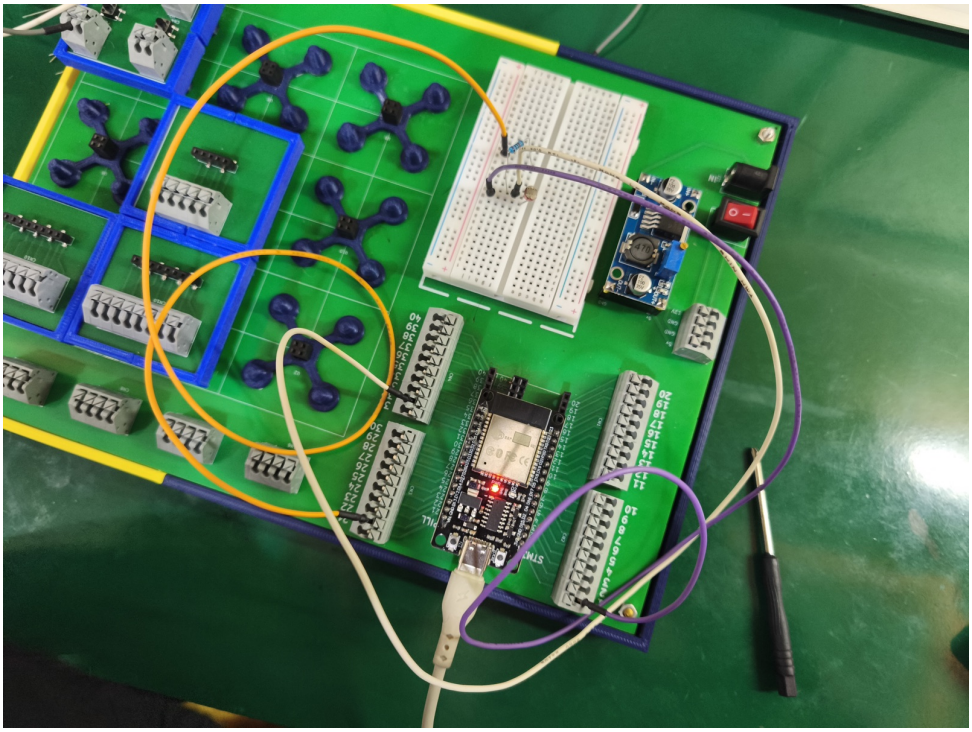
1. Merangkai LDR sebagai pembagi tegangan: satu kaki LDR ke 3,3 V, kaki lainnya ke GPIO 34 dan ke GND lewat resistor 10 kΩ (Gambar 16).

2. Menulis program yang membaca `analogRead()` pada GPIO 34 lalu mengubahnya menjadi tegangan dengan $V = \text{raw} \times 3{,}3/4095$.

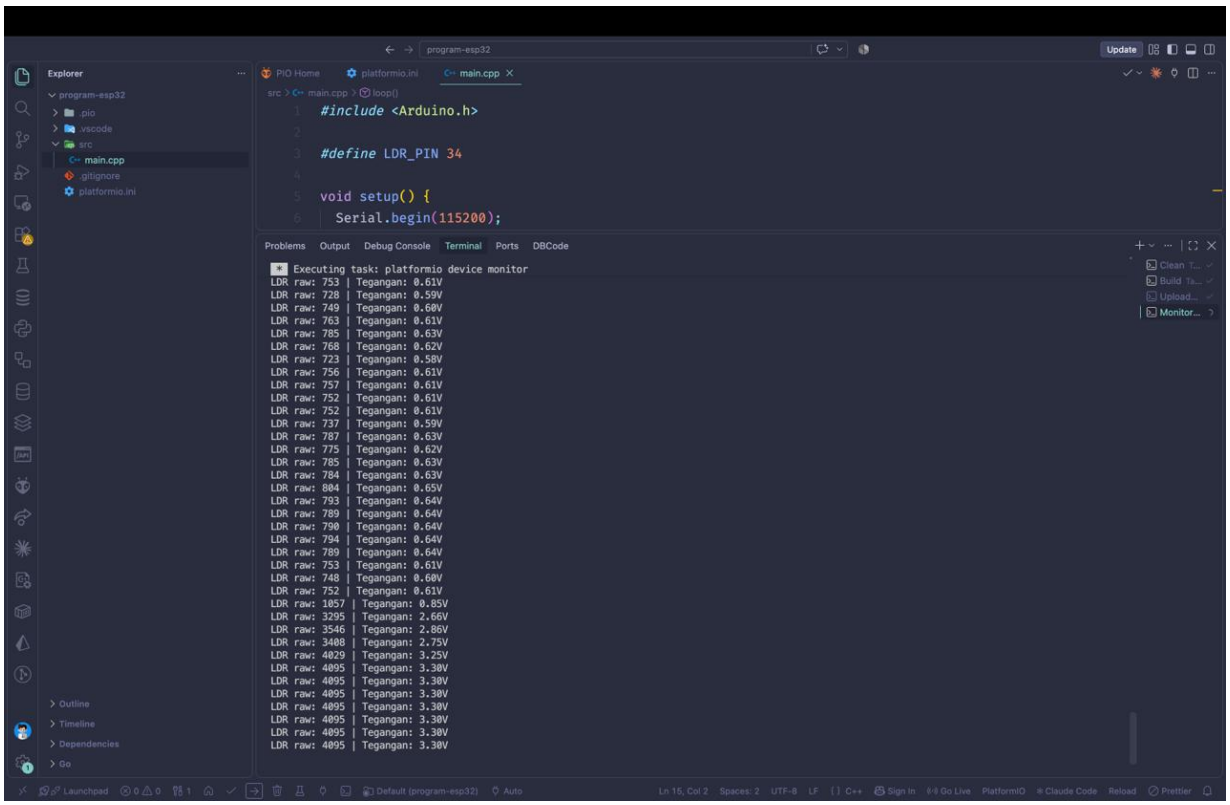
3. Menjalankan *Build* dan *Upload*, lalu membuka Serial Monitor (Gambar 17).

4. Menutup LDR dengan tangan, lalu menyinarinya dengan lampu (Gambar 18), sambil mengamati nilai mentah 0 sampai 4095 yang berubah.

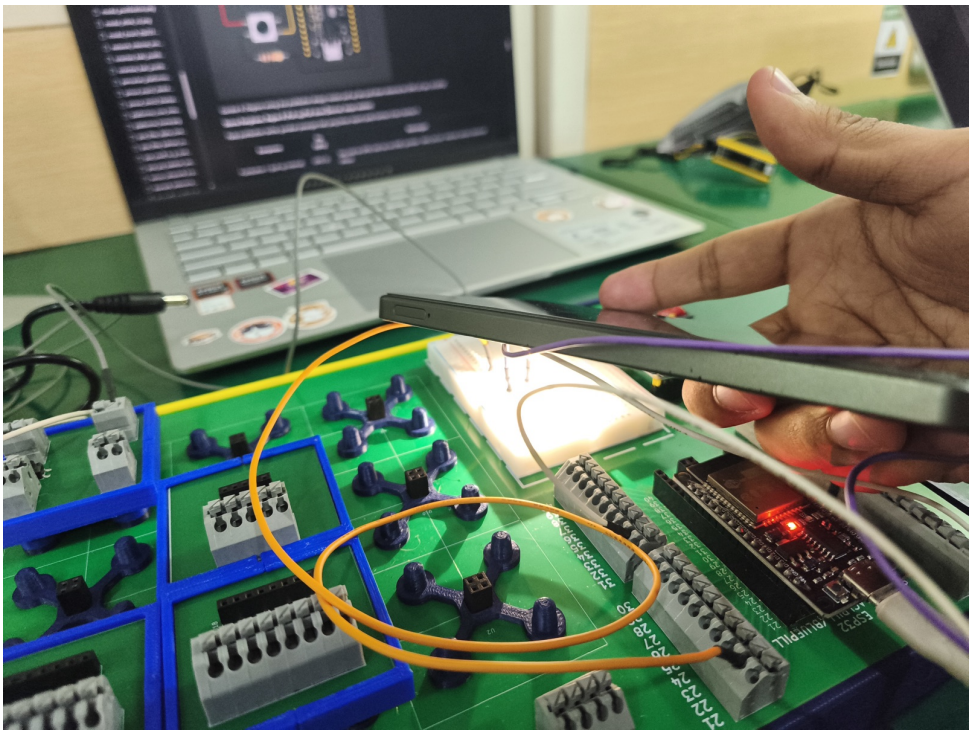
5. Menganalisis mengapa sinyal analog dapat menunjukkan banyak kondisi, tidak hanya HIGH atau LOW.



Gambar 16: Rangkaian LDR dan resistor 10 kΩ ke GPIO 34



Gambar 17: Serial Monitor nilai ADC dan tegangan LDR



Gambar 18: LDR disinari cahaya saat pengujian

2 Analisis Hasil Percobaan

2.1 Percobaan 1: STM32 Blackpill (Blink)

Program berhasil dikompilasi dan diunggah lewat ST-Link, lalu LED di pin PC13 berkedip dengan selang 500 ms menyala dan 500 ms mati. Ini menunjukkan konfigurasi `platformio.ini` untuk Blackpill (STM32F401CCU6, Cortex-M4) sudah benar, dan keempat kabel SWD (SWDIO, SWCLK, GND, 3,3 V) tersambung dengan baik, sebab ST-Link hanya bisa memprogram target jika jalur itu lengkap.

LED onboard Blackpill bersifat aktif LOW. Karena itu `digitalWrite(PC13, LOW)` menyalakan LED dan `HIGH` mematikannya, yang kebalikan dari kebiasaan pada LED eksternal. Penamaan pin langsung sesuai label port seperti `PC13` dimungkinkan oleh STM32duino, sehingga struktur konfigurasi HAL tidak perlu ditulis manual.

2.2 Percobaan 2: ESP32 dengan ESP-IDF (Blink)

Program C pada ESP-IDF berhasil diunggah lewat USB dan LED pada GPIO 2 berkedip. Port serial terdeteksi tanpa programmer tambahan karena board DevKit sudah membawa chip USB-to-UART. Perbandingan dengan Arduino yang tampak dari praktikum ini:

Aspek	Arduino	ESP-IDF
Titik masuk	`setup()` dan `loop()`	`app_main()` sebagai *task* FreeRTOS
Penundaan	`delay()`	`vTaskDelay(pdMS_TO_TICKS())`, *task* lain tetap bisa jalan
Konfigurasi pin	`pinMode()`, `digitalWrite()`	`gpio_reset_pin()`, `gpio_set_direction()`, `gpio_set_level()`
Berkas program	`main.cpp`	`main.c` (bahasa C)
Abstraksi	Tinggi, ringkas	Lebih rendah, kontrol lebih rinci

Tabel 2: Perbandingan pemrograman Arduino dan ESP-IDF pada Percobaan 2

Untuk program sekecil Blink, ESP-IDF terasa lebih panjang karena pin harus disiapkan dan ditunda lewat API RTOS. Keuntungannya baru terlihat pada program yang memakai banyak *task*, karena penjadwalan FreeRTOS bisa diatur langsung. Kompilasi pertama ESP-IDF juga cenderung lebih lama karena komponen RTOS dan pustaka sistem ikut dibangun.

2.3 Percobaan 3: Pull-up dan Pull-down

Serial Monitor pada Kode Program A (resistor eksternal) dan Kode Program B (resistor internal) menunjukkan perilaku yang sama. Pada konfigurasi *pull-up* (GPIO 32), pin terbaca HIGH saat tombol dilepas dan LOW saat ditekan, sehingga kodenya menuliskan “DITEKAN” ketika `digitalRead()` bernilai LOW. Pada konfigurasi *pull-down* (GPIO 33), logikanya terbalik: LOW saat dilepas dan HIGH saat ditekan.

Pin	Konfigurasi	Dilepas	Ditekan
GPIO 32	*pull-up* (eksternal atau internal)	HIGH	LOW
GPIO 33	*pull-down* (eksternal atau internal)	LOW	HIGH

Tabel 3: Level logika yang teramati pada Percobaan 3

Pada keluaran Kode Program B terlihat semua kombinasi muncul sesuai tombol yang ditekan: hanya tombol 1 ditekan, hanya tombol 2 ditekan, keduanya ditekan, dan keduanya dilepas. Hasil yang identik antara A dan B menunjukkan resistor internal ESP32 cukup untuk menggantikan resistor eksternal pada rangkaian tombol sederhana ini, sehingga komponen dan kabel bisa dihemat. Resistor internal ESP32 bernilai sekitar 45 kΩ menurut datasheet ESP32, lebih besar daripada 10 kΩ yang dipakai eksternal, sehingga penarikannya lebih lemah dan lebih mudah terganggu derau jika kabelnya panjang. Selain itu GPIO 34 sampai 39 tidak punya resistor internal, jadi pin tersebut tidak bisa dipakai untuk Bagian B.

2.4 Percobaan 4: Debouncing

Pada keluaran Serial Monitor (Gambar 15) `counterNoDebounce` selalu naik lebih dulu, sebab ia bertambah seketika saat tepi turun terdeteksi, sedangkan `counterWithDebounce` baru naik setelah sinyal bertahan stabil 50 ms. Pola ini terlihat dari pasangan nilai yang berselang-seling, misalnya $(1, 0)$ lalu $(1, 1)$, kemudian $(2, 1)$ lalu $(2, 2)$.

Efek *bouncing* terlihat pada baris-baris berikut:

Tanpa debounce	Dengan debounce	Keterangan
9	9	Keadaan awal sebelum penekanan ke-10
10	9	Tepi turun pertama terdeteksi
11	9	Tepi turun kedua terdeteksi pada tekanan yang sama (pantulan)
11	10	Setelah 50 ms stabil, hanya dihitung satu tekanan

Tabel 4: Cuplikan penghitung pada tangkapan layar Gambar 15

Satu tekanan menaikkan penghitung mentah dua kali, tetapi penghitung ber-*debounce* hanya naik satu. Selisih satu hitungan ini bertahan sampai akhir tangkapan (penghitung mentah 17, ber-*debounce* 15, dengan satu tekanan terakhir yang masih menunggu 50 ms), yang menegaskan bahwa pantulan kontak memang terjadi dan tersaring oleh algoritma. Pantulan tidak muncul pada setiap tekanan, jadi sebagian besar tekanan menaikkan kedua penghitung dengan sama.

Pengaruh besar `DEBOUNCE_DELAY` dianalisis dari cara kerja algoritmanya. Pada 10 ms, jeda bisa lebih pendek dari lama pantulan sehingga sebagian pantulan berpotensi lolos dan terhitung ganda, meski tombol terasa paling cepat. Pada 50 ms, jeda sudah melampaui pantulan yang umum terjadi dan tombol masih terasa responsif, sehingga ini nilai yang seimbang. Pada 200 ms, pantulan pasti tersaring, tetapi tekanan cepat yang berselang kurang dari 200 ms akan terlewat karena sinyal belum sempat stabil, sehingga tombol terasa lambat. Jadi makin besar nilainya makin bersih dari getaran, dengan harga respons yang makin lambat.

2.5 Percobaan 5: ADC dengan LDR

Nilai ADC berubah mengikuti cahaya yang mengenai LDR. Pada kondisi awal sebelum disinari, pembacaan berada di sekitar 723 sampai 804 (sekitar 0,58 sampai 0,65 V). Ketika LDR disinari dengan lampu, nilainya naik cepat melalui 1057 (0,85 V), 3295 (2,66 V), 3546 (2,86 V), 3408 (2,75 V), dan 4029 (3,25 V), lalu mencapai 4095. Konversinya memakai $V = \text{raw} \times 3{,}3/4095$, dengan resolusi

$$\Delta V = \frac{3{,}3}{4095} \approx 0{,}806 \text{ mV per langkah.} \tag{1}$$

Dari rumus pembagi tegangan $V_o = V_{in}\,R/(R_{LDR} + R)$ dengan $R = 10$ kΩ dan $V_{in} = 3{,}3$ V, hambatan LDR dapat diperkirakan dari tegangan terukur, yaitu $R_{LDR} = R\,(V_{in}/V_o - 1)$:

ADC	V_o (V)	R_{LDR} perkiraan
753	0,61	44 kΩ
1057	0,85	29 kΩ
3295	2,66	2,4 kΩ
3546	2,86	1,5 kΩ

Tabel 5: Perkiraan hambatan LDR dari pembacaan ADC

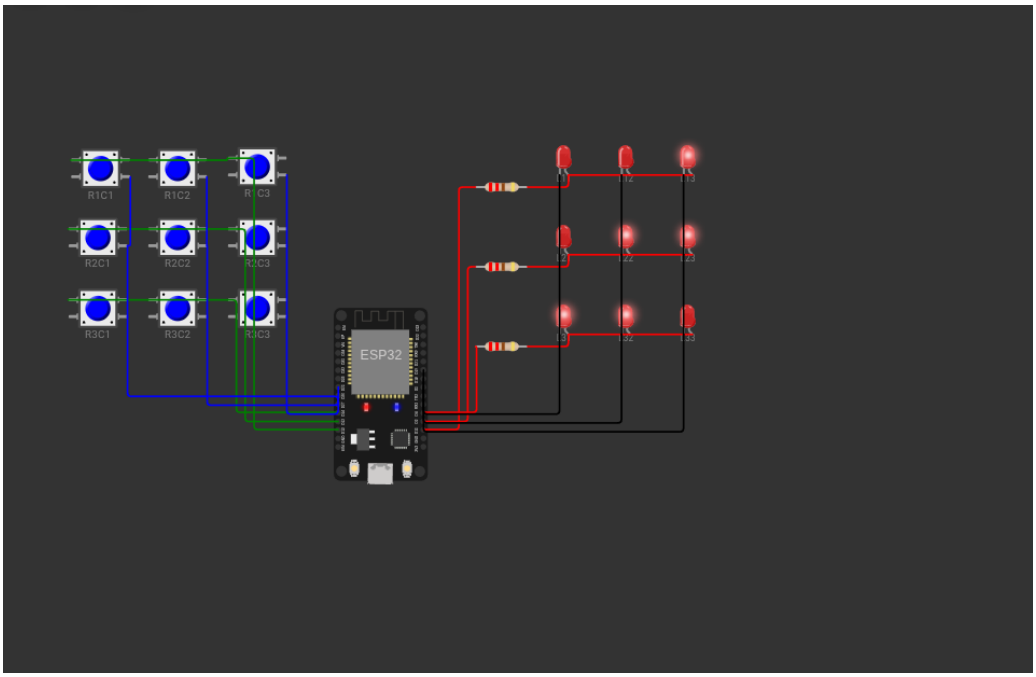
Hambatan LDR turun dari puluhan kilo-ohm menjadi satu sampai dua kilo-ohm ketika terang, sesuai sifat LDR yang hambatannya mengecil saat cahaya bertambah. Nilai 4095 berarti ADC sudah mencapai batas atas. Tegangan 3,30 V pada baris itu hanya hasil rumus, karena ADC ESP32 tidak linear sempurna di dekat batas atas dan dapat jenuh sebelum 3,3 V, sehingga angka pada kisaran terang sebaiknya dibaca sebagai perkiraan, bukan ukuran yang presisi.

Sinyal analog dapat menunjukkan banyak kondisi karena tegangan pada titik tengah pembagi berubah halus mengikuti hambatan LDR, lalu ADC 12 bit membaginya menjadi 4096 tingkat. Masukan digital hanya mengelompokkan tegangan menjadi dua kelas, di bawah atau di atas ambang, sehingga gradasi gelap, redup, dan terang tidak terbedakan.

3 Hasil Tugas Modul

3.1 Matrix Pushbutton 3×3 Mengendalikan Matrix LED 3×3

Tugas meminta simulasi matrix pushbutton 3×3 yang mengendalikan matrix LED 3×3: tombol pada baris i kolom j mengubah LED pada baris i kolom j, dan setiap tombol wajib ber-*debouncing*. Simulasi dibuat di Wokwi dengan ESP32 (Gambar 19).



Gambar 19: Simulasi matrix pushbutton 3×3 dan matrix LED 3×3 pada Wokwi

Rancangan. Tombol dipindai per baris: satu baris ditarik LOW pada satu waktu, lalu keadaan tiga kolom dibaca dengan `INPUT_PULLUP`. Tombol yang ditekan menghubungkan kolomnya ke baris aktif sehingga kolom terbaca LOW. LED disusun sebagai matrix dengan anoda per baris dan katoda per kolom, lalu dinyalakan bergantian per baris (*multiplexing*) cukup cepat agar tampak menyala bersamaan. Setiap tombol memiliki pengukur waktu *debounce* sendiri sebesar 50 ms, dan LED berganti status (*toggle*) hanya saat tombol berpindah ke LOW secara stabil.

Fungsi	GPIO ESP32	Mode
Baris tombol 1, 2, 3	13, 12, 14	Keluaran (pemindaian)
Kolom tombol 1, 2, 3	27, 26, 25	Masukan `INPUT_PULLUP`
Anoda LED baris 1, 2, 3	15, 2, 4	Keluaran, aktif HIGH
Katoda LED kolom 1, 2, 3	5, 18, 19	Keluaran, aktif LOW

Tabel 6: Pemetaan pin pada simulasi

Kode program.

```
#include <Arduino.h>

void scanButtonsWithDebounce();
void refreshLedMatrix();

const int btnRowPins[3] = {13, 12, 14}; // output pemindaian baris
const int btnColPins[3] = {27, 26, 25}; // input kolom (INPUT_PULLUP)
const int ledRowPins[3] = {15, 2, 4};   // anoda LED, aktif HIGH
const int ledColPins[3] = {5, 18, 19};  // katoda LED, aktif LOW

bool buttonState[3][3];               // status stabil tombol
bool lastButtonReading[3][3];         // pembacaan mentah sebelumnya
unsigned long lastDebounceTime[3][3]; // waktu terakhir sinyal berubah
const unsigned long debounceDelay = 50; // ms
bool ledState[3][3] = {{false, false, false},
                       {false, false, false},
                       {false, false, false}};

void setup() {
  Serial.begin(115200);
  for (int r = 0; r < 3; r++) {
    pinMode(btnRowPins[r], OUTPUT);
    digitalWrite(btnRowPins[r], HIGH);
    pinMode(ledRowPins[r], OUTPUT);
    digitalWrite(ledRowPins[r], LOW);
  }
  for (int c = 0; c < 3; c++) {
    pinMode(btnColPins[c], INPUT_PULLUP);
    pinMode(ledColPins[c], OUTPUT);
    digitalWrite(ledColPins[c], HIGH);
  }
  for (int r = 0; r < 3; r++)
    for (int c = 0; c < 3; c++) {
      buttonState[r][c] = HIGH;
      lastButtonReading[r][c] = HIGH;
      lastDebounceTime[r][c] = 0;
    }
  Serial.println("Sistem Matrix 3x3 siap. Tekan tombol untuk toggle LED.");
```

```
39 }
40
41 void loop() {
42   scanButtonsWithDebounce();
43   refreshLedMatrix();
44 }
45
46 void scanButtonsWithDebounce() {
47   for (int r = 0; r < 3; r++) {
48     digitalWrite(btnRowPins[r], LOW); // aktifkan baris r
49     for (int c = 0; c < 3; c++) {
50       bool reading = digitalRead(btnColPins[c]);
51       if (reading != lastButtonReading[r][c])
52         lastDebounceTime[r][c] = millis(); // sinyal berubah: reset timer
53       if ((millis() - lastDebounceTime[r][c]) > debounceDelay) {
54         if (reading != buttonState[r][c]) {
55           buttonState[r][c] = reading;
56           if (buttonState[r][c] == LOW) { // tombol stabil ditekan
57             ledState[r][c] = !ledState[r][c];
58             Serial.printf("Tombol [Baris %d, Kolom %d] ditekan -> LED %s\n",
59                           r + 1, c + 1, ledState[r][c] ? "ON" : "OFF");
60           }
61         }
62       }
63       lastButtonReading[r][c] = reading;
64     }
65     digitalWrite(btnRowPins[r], HIGH); // kembalikan baris r
66   }
67 }
68
69 void refreshLedMatrix() {
70   for (int r = 0; r < 3; r++) {
71     for (int c = 0; c < 3; c++)
72       digitalWrite(ledColPins[c], ledState[r][c] ? LOW : HIGH);
73     digitalWrite(ledRowPins[r], HIGH);
74     delayMicroseconds(1500);          // tahan 1,5 ms per baris
75     digitalWrite(ledRowPins[r], LOW); // matikan sebelum pindah (cegah ghosting)
76   }
77 }
```

Pada `main.cpp` di PlatformIO, dua fungsi di bawah `loop()` harus dideklarasikan dulu di atas (baris 3 dan 4). Pada Wokwi dengan berkas `.ino` hal ini tidak diperlukan.

3.2 Jawaban Pertanyaan Analisis

1. **Mengapa debouncing tetap diperlukan pada matrix pushbutton walaupun biasanya hanya satu tombol ditekan?**

   Bouncing tidak bergantung pada jumlah tombol yang ditekan, melainkan pada sifat mekanis setiap tombol. Pelat pegas logam di dalam tombol memantul beberapa kali sebelum menempel atau lepas sempurna (C.5), sehingga selama beberapa milidetik sinyal berganti HIGH dan LOW berulang. ESP32 memindai ribuan kali per detik, jadi ia menangkap semua pantulan itu. Tanpa *debouncing*, satu tekanan terbaca sebagai beberapa tekanan. Karena program ini mengubah status LED se-

cara *toggle*, jumlah pantulan yang genap membuat LED kembali ke status semula, sedangkan jumlah ganjil membuatnya berubah, jadi hasilnya tidak bisa diprediksi. Dengan syarat sinyal stabil 50 ms yang dihitung per tombol, satu tekanan hanya menghasilkan satu perubahan status.

2. **Konfigurasi pull-up atau pull-down pada pembacaan kolom, kondisi pin saat tombol tidak ditekan, dan mengapa tidak boleh floating?**

   Rancangan memakai *pull-up* internal (`INPUT_PULLUP`) pada kolom GPIO 27, 26, dan 25, sedangkan baris menjadi sumber sinyal aktif LOW. Saat tombol tidak ditekan, pin kolom ditarik ke 3,3 V oleh resistor internal sehingga terbaca HIGH. Saat tombol di persimpangan baris yang sedang aktif ditekan, kolom tersambung ke baris yang bernilai LOW sehingga terbaca LOW. Pin tidak boleh dibiarkan *floating* karena pin berimpedansi tinggi tanpa referensi tegangan mudah terpengaruh derau elektromagnetik, listrik statis, dan kapasitansi parasit kabel jumper. Pembacaannya bisa berubah acak antara HIGH dan LOW padahal tombol tidak disentuh, yang membuat LED berganti status sendiri.

3. **Perbedaan membaca GPIO sebagai digital input dan analog input (ADC), dan mengapa ADC dapat merepresentasikan lebih dari dua kondisi?**

   Pada masukan digital, rangkaian pembanding pada pin hanya mengelompokkan tegangan menjadi dua kelas: di bawah ambang terbaca LOW dan di atas ambang terbaca HIGH. Besar tegangan di dalam masing-masing kelas tidak diketahui. Pada masukan analog, ADC mengukur besar tegangan kontinu pada rentang 0 sampai 3,3 V dan mengubahnya menjadi bilangan bulat lewat kuantisasi. ADC 12 bit pada ESP32 membagi rentang itu menjadi 4096 tingkat (0 sampai 4095), dengan langkah sekitar 0,8 mV. Jika tombol dalam matrix diganti LDR, hambatan LDR berubah bertahap mengikuti cahaya sehingga tegangan titik tengah pembagi ikut berubah halus, dan ADC dapat membedakan gelap, redup, terang, sampai sangat terang. Tombol hanya bisa mewakili dua kondisi, ditekan atau tidak.

4 Kesimpulan

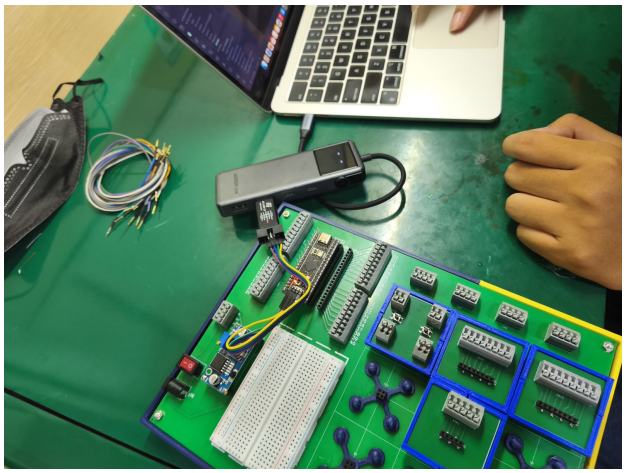
Dari Modul 1 dapat disimpulkan bahwa lingkungan PlatformIO di VSCode berhasil dipakai untuk memprogram dua jenis mikrokontroler. STM32 Blackpill diprogram lewat ST-Link dengan framework Arduino, dan ESP32 diprogram lewat USB, baik dengan ESP-IDF maupun Arduino. Pilihan framework menentukan tingkat kendali: Arduino ringkas dan cepat untuk memulai, sedangkan ESP-IDF memberi akses langsung ke FreeRTOS dan perangkat keras dengan kode yang lebih panjang.

Pada masukan digital, resistor *pull-up* dan *pull-down*, eksternal maupun internal, memberi hasil logika yang sama dan mencegah pin *floating*. Pada Percobaan 4, *bouncing* terbukti menaikkan penghitung mentah lebih dari satu untuk satu tekanan, dan *debouncing* perangkat lunak dengan jeda 50 ms menyaringnya tanpa membuat tombol terasa lambat. Pada Percobaan 5, ADC 12 bit mengubah tegangan kontinu dari LDR menjadi 4096 tingkat, sehingga gradasi cahaya dapat dibedakan, sesuai dengan sifat LDR yang hambatannya turun saat cahaya bertambah. Tugas matrix pushbutton dan LED 3×3 menggabungkan semuanya: pemindaian baris-kolom dengan *pull-up* internal, *debouncing* per tombol, dan *multiplexing* LED, dan berjalan sesuai rancangan pada simulasi.

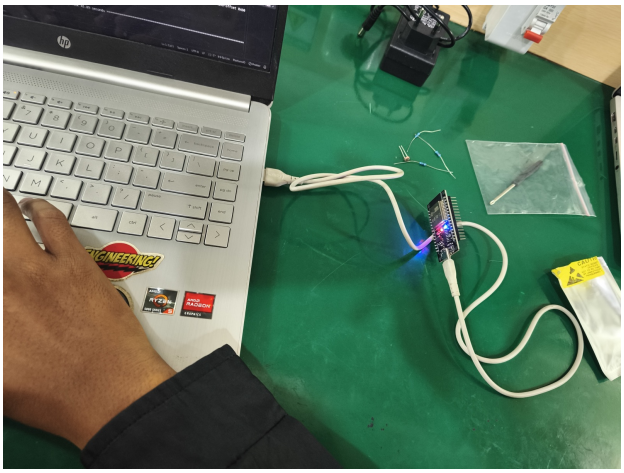
5 Lampiran

5.1 Dokumentasi saat praktikum

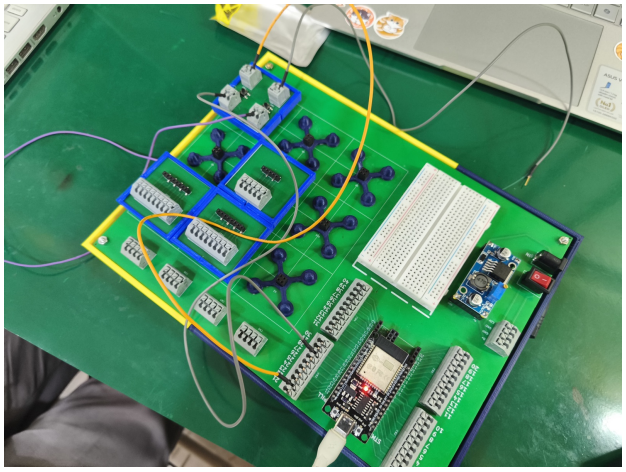
Foto berikut memperlihatkan alat yang dipakai dan kegiatan selama praktikum.



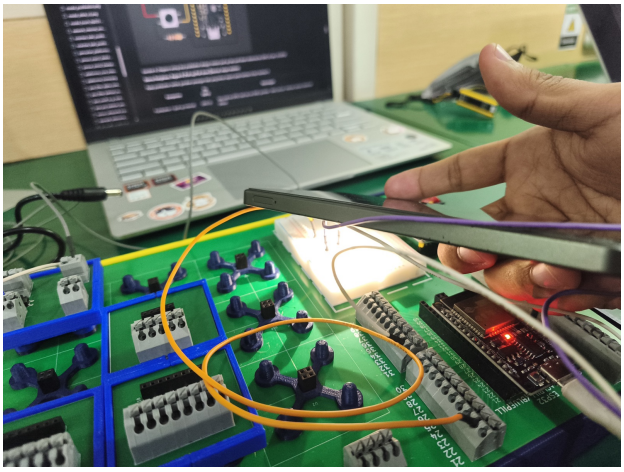
Gambar 20: Pemrograman Blackpill dengan ST-Link



Gambar 21: Pemrograman ESP32 lewat USB



Gambar 22: Rangkaian tombol pada Percobaan 4



Gambar 23: Pengujian LDR pada Percobaan 5